

2D SPACE SHOOTER NETCODE



Overview

- Goal: Understand how this 2D co-op space shooter uses Unity Netcode for GameObjects to run in multiplayer.
- Unity version: 6000.0.53f1
- Repository url: <https://github.com/Unity-Technologies/com.unity.multiplayer.samples.bitesize/tree/main/Basic/2DSpaceShooter>

What you'll learn

- The basic Netcode building blocks used here: NetworkManager, NetworkObject, NetworkBehaviour, NetworkVariable, RPCs.
- How authority works: the server runs gameplay logic, clients send input and render feedback.
- How objects are spawned, pooled, and synchronized.
- How UI and connection flow are handled.

TABLE OF CONTENTS

2D SPACE SHOOTER NETCODE	1
Overview	1
What you'll learn	1
KEY MULTIPLAYER BUILDING BLOCKS IN THIS PROJECT	3
NetworkManager + Transport	3
NetworkObject and NetworkBehaviour	3
NetworkVariable	3
RPCs (Remote Procedure Calls)	3
Authority and Ownership	3
Prefab Pooling for NetworkObjects	3
Spawning and Connection Approval	3
Server vs Client Visuals	4
FILE-BY-FILE GUIDE.....	4
1) Network UI and flow: NetworkManagerHud.cs	4
2) Player spawn positions: RandomPositionPlayerSpawner.cs	4
3) Pooling networked prefabs: NetworkObjectPool.cs (+ DummyPrefabInstanceHandler)	4
4) The player ship: ShipControl.cs	5
5) Bullets: Bullet.cs	6
6) Asteroids: Asteroid.cs	6
7) Powerups: Powerup.cs	7
8) World content spawning: Spawner.cs	7
9) VFX pooling (local only): ExplosionsPool.cs and ReturnParticleSystemToPool.cs	7
AUTHORITY AND TIMING	8
TYPICAL GAMEPLAY FLOWS	8
1) Movement	8
2) Firing	8
3) Powerup pickup	9
4) Asteroid destruction.....	9
CONCEPT GLOSSARY	9
THINGS TO TRY (EXERCISES).....	10
WHERE TO LOOK FOR EACH TOPIC	10

KEY MULTIPLAYER BUILDING BLOCKS IN THIS PROJECT

NetworkManager + Transport

- Central networking singleton (NetworkManager.Singleton).
- Uses Unity Transport (UTP) for host/server/client connections, IP and port configuration, and connection events.

NetworkObject and NetworkBehaviour

- NetworkObject is required on networked prefabs (e.g., ships, bullets, asteroids, powerups).
- NetworkBehaviour is a MonoBehaviour that participates in networking (e.g., ShipControl, Bullet, Asteroid, Powerup).

NetworkVariable

- Sync small pieces of state automatically. The server writes; all clients read and receive updates.
- Examples: player health/energy, buff timers, color, names, asteroid size, powerup type.

RPCs (Remote Procedure Calls)

- Methods marked with [Rpc(...)] that run on remote peers.
- Server-bound RPCs: clients send input/commands to the server (e.g., thrust, fire).
- Client-bound RPCs: server tells clients to do client-side feedback (e.g., play a sound, update velocity).

Authority and Ownership

- Server authority: the server is the source of truth for gameplay (physics, damage, spawning).
- Ownership: each player “owns” their ship. Only the owning client sends input for that ship.
- IsServer, IsClient, IsHost, IsLocalPlayer checks control what runs where.

Prefab Pooling for NetworkObjects

- Reuse objects instead of creating/destroying repeatedly (performance and GC friendly).
- Integrates with Netcode via INetworkPrefabInstanceHandler so pooled instances participate correctly in spawn/despawn.

Spawning and Connection Approval

- Server decides when to Spawn() or Despawn() network objects.

- Connection approval callback used to assign a spawn position for new players.

Server vs Client Visuals

- The server simulates gameplay; clients handle visuals like UI, camera, and particles.
- Some effects (e.g., explosion particles) are intentionally client-local and not networked.

FILE-BY-FILE GUIDE

1) Network UI and flow: `NetworkManagerHud.cs`

- Role: Provides a simple main menu to Start Host, Start Server, Start Client, and to Shutdown. Manages connection status UI.
- How it uses Netcode:
 - Gets the `NetworkManager` and `UnityTransport`.
 - When Host/Server/Client buttons are pressed, it starts the corresponding mode.
 - Subscribes to `OnConnectionEvent` to switch UI between menu and in-game views.

2) Player spawn positions: `RandomPositionPlayerSpawner.cs`

- Role: Assigns a spawn position for each connecting player (random or round-robin).
- How it uses Netcode:
 - Hooks into `NetworkManager.ConnectionApprovalCallback`.
 - Approves all connections but sets `response.Position` and `response.Rotation` for the player object.

3) Pooling networked prefabs: `NetworkObjectPool.cs` (+

`DummyPrefabInstanceHandler`)

- Role: Central pool for networked prefabs (e.g., bullets, asteroids, powerups).
- How it uses Netcode:
 - Validates that pooled prefabs have `NetworkObject` components.
 - Registers a handler with `NetworkManager.PrefabHandler.AddHandler(prefab, handler)`.
 - Handler implements `INetworkPrefabInstanceHandler` to integrate pooling with Netcode spawn/despawn.
 - `GetNetworkObject` returns pooled or newly created instances; `ReturnNetworkObject` disables and enqueues them.

4) The player ship: `ShipControl.cs`

- Role: Core player controller with movement, firing, buffs, UI, and health/energy.
- How it uses Netcode:
 - Inherits `NetworkBehaviour`. Uses many `NetworkVariables`:
 - Health, Energy
 - Buff timers (`SpeedBuffTimer`, `RotateBuffTimer`, `TripleShotTimer`, `DoubleShotTimer`, `QuadDamageTimer`, `BounceTimer`)
 - LatestShipColor
 - PlayerName (`FixedString32Bytes`)
 - Client sends input to server:
 - `ServerThrustRpc(thrust, spin)`: Client → Server. Server updates physics (thrust and rotation).
 - `ServerFireRpc()`: Client → Server. Server checks Energy and fires bullets (spawns them via the pool).
 - `ServerSetNameRpc(name)`: Client → Server. Server sets PlayerName.
 - Server runs the authoritative simulation:
 - Energy regeneration, movement forces, top speed limits, collisions with asteroids, respawning on death.
 - Client-only responsibilities:
 - If `IsLocalPlayer`, centers the camera on your own ship.
 - Updates UI (health/energy bars, player name).
 - Plays client-side fire sound via `ClientPlayFireSoundRpc()`.
- Why it matters:
 - Demonstrates the classic multiplayer pattern:
 - Client reads local input, sends minimal commands to the server.
 - Server applies physics and game logic, updates shared state (`NetworkVariables`), and spawns objects.
 - Clients render visuals and UI based on replicated state.

5) Bullets: `Bullet.cs`

- Role: Projectile logic including lifetime, collisions, and client-side effects.
- How it uses Netcode:
 - Inherits `NetworkBehaviour`.
 - Server-only configuration and lifetime:
 - `Config(owner, damage, bounce, lifetime)` and timed despawn run on the server.
 - Velocity replication:
 - Server sets `Rigidbody2D.linearVelocity` and calls a client-bound RPC to mirror velocity: `ClientSetVelocityRpc()`.
 - The host check avoids double-applying on the host.
 - Collisions:
 - `OnCollisionEnter2D` runs on the server to apply damage or destroy itself.
 - Client visuals:
 - `OnNetworkDespawn` on clients plays an explosion particle effect (non-networked, pooled locally).
- Why it matters:
 - Shows how to keep authoritative physics on the server while keeping clients visually in sync.

6) Asteroids: `Asteroid.cs`

- Role: Networked asteroids that can split when destroyed.
- How it uses Netcode:
 - Inherits `NetworkBehaviour`, uses a `NetworkVariable Size` to scale visuals.
 - `Explode()` runs server-side:
 - Decrements asteroid count, optionally spawns smaller asteroids using the `NetworkObjectPool`.
 - For each child asteroid: set `Size`, copy prefab reference, `Spawn()`, and apply forces.
 - Despawn via `NetworkObject.Despawn(true)` on the server to remove from all clients.

7) Powerups: `Powerup.cs`

- Role: Networked pick-ups that apply buffs to the ship that collects them.
- How it uses Netcode:
 - Inherits `NetworkBehaviour`, has `NetworkVariable<Buff.BuffType> buffType` for synchronized type/color/UI text.
 - `OnNetworkSpawn` hooks `OnValueChanged` to update visuals.
 - `OnTriggerEnter2D`: Server checks for a `ShipControl` and applies `AddBuff(buffType.Value)`, then `Despawn(true)`.
 - Client-only UI positioning uses UI Toolkit and camera space calculations.
- Why it matters:
 - Clear example of server-side game rule (grant the buff, then despawn), client-side visual updates, and `NetworkVariable`-driven presentation.

8) World content spawning: `Spawner.cs`

- Role: Spawns static obstacles locally and manages server-side spawning of powerups and asteroids.
- How it uses Netcode:
 - Obstacles: Instantiated locally on all peers (not networked).
 - Powerups: On the server, uses the pool to get an instance, sets position, `Spawn(true)`, and assigns a random buff type through a `NetworkVariable`.
 - Asteroids: If none remain, server spawns a new wave via pooling and `Spawn(true)`.
- Why it matters:
 - Teaches when to network an object (things that move or change and must be shared) vs. when to instantiate locally (static environment).

9) VFX pooling (local only): `ExplosionsPool.cs` and `ReturnParticleSystemToPool.cs`

- Role: Efficiently reuse particle systems for explosions on each client.
- How it uses Netcode:
 - These components don't use Netcode directly.
 - Bullets trigger explosion visuals on clients in `OnNetworkDespawn`; each client plays its own effect from a local pool.
- Why it matters:
 - A common optimization: keep expensive VFX local. Not everything needs networking.

AUTHORITY AND TIMING

- Server authority:
 - Movement physics and energy regen happen on the server (ShipControl.UpdateServer).
 - Collisions that change game state (bullet hits, asteroid splits, powerup pickup) run on the server.
- Client responsibilities:
 - Reading local input and sending commands to the server (ServerThrustRpc, ServerFireRpc).
 - UI, audio, and camera follow for the local player.
- Time sources:
 - ServerTime is used to schedule/regulate buffs and regeneration consistently.
 - LocalTime can be used for client-side effects, but gameplay decisions use ServerTime.

TYPICAL GAMEPLAY FLOWS

1) Movement

- Client (local player):
 - Reads input, computes thrust/spin values.
 - Sends ServerThrustRpc(thrust, spin).
- Server:
 - Applies forces/rotation to the authoritative Rigidbody2D.
 - Updates shared NetworkVariables as needed.
- Clients:
 - Render updated positions from server authority.

2) Firing

- Client:
 - Detects fire input and calls ServerFireRpc().
- Server:
 - Checks Energy and active buff timers.
 - Spawns one or more bullets via the NetworkObjectPool and calls Spawn(true).
 - Configures bullet lifetime, damage, bounce, and initial velocity.
 - Optionally notifies clients to play local audio via Client RPCs.
- Clients:
 - Play fire sound. Receive bullet velocity via ClientSetVelocityRpc() and update local Rigidbody2D for smooth visuals.

3) Powerup pickup

- Server:
 - OnTriggerEnter2D detects a ship, applies AddBuff(type), then Despawn(true) the powerup.
- Clients:
 - UI updates automatically as buff timers change through NetworkVariables.

4) Asteroid destruction

- Server:
 - Asteroid.Explode() reduces size; if still > 0, spawns smaller asteroids with Spawn().
 - Despawns the original asteroid.
- Clients:
 - See new asteroids spawn; when bullets/asteroids despawn, local particles are played.

CONCEPT GLOSSARY

- NetworkManager: The singleton that runs and configures your multiplayer session.
- Unity Transport (UTP): The network transport layer used by Netcode in this sample.
- NetworkObject: Component required on any prefab that will be spawned/synchronized over the network.
- NetworkBehaviour: Base class for scripts that participate in networking (access to IsServer, IsClient, IsHost, OwnerClientId, RPCs, etc.).
- NetworkVariable: Synchronized variable. Server writes; clients read and get change notifications.
- RPC: Remote method invocation. In this sample, methods are tagged with [Rpc(SendTo.Server)] or [Rpc(SendTo.ClientsAndHost)].
- Spawn/Despawn: Server-controlled creation and removal of networked instances so all clients stay in sync.
- INetworkPrefabInstanceHandler: Interface to integrate your pooling logic with Netcode's spawning system.
- ServerTime vs LocalTime: Use ServerTime for authoritative gameplay timing.

THINGS TO TRY (EXERCISES)

- Add a new buff type and have ShipControl respond to it with a new effect.
- Make bullets move faster only for a short time using a new NetworkVariable timer.
- Add a UI text for the player's name and let clients set it via ServerSetNameRpc().
- Spawn more asteroids per wave and observe how the pool helps performance.
- Change RandomPositionPlayerSpawner to always spawn near the center—see how the client ships appear to each other.

WHERE TO LOOK FOR EACH TOPIC

- Starting/stopping networking: NetworkManagerHud.cs
- Spawning players at positions: RandomPositionPlayerSpawner.cs
- Pooling networked objects: NetworkObjectPool.cs
- Player control and RPC patterns: ShipControl.cs
- Projectiles and client-side visuals: Bullet.cs + ExplosionsPool.cs
- Enemy/networked actors: Asteroid.cs
- Pickups and synchronized visuals: Powerup.cs
- World content: Spawner.cs